

EMBEDDED SYSTEMS INTERNSHIP

WEEK 1 PROJECT REPORT

System Requirements Analysis and Block Diagram Design

Project: IoT-Based Smart Environmental Monitoring and Alert System

Prepared For	Embedded Systems / IoT Internship – Week 1
Project Type	Hypothetical embedded device with sensing, local alerting, actuation and cloud connectivity
Primary Controller	ESP32-class microcontroller
Target Effort	Approx. 30–35 hours of analysis, architecture, documentation and validation planning

Professional submission document covering requirements, architecture, interfaces, power, real-time behavior, software structure, testing and future scalability.

1. Abstract

This project defines the requirements and architecture of an IoT-based smart environmental monitoring and alert system. The device continuously measures temperature, humidity, ambient light, air-quality indicators and motion. An ESP32-class microcontroller acquires and processes the sensor signals, drives a local OLED display, activates a buzzer and relay when thresholds are exceeded, and publishes selected measurements to a remote dashboard using Wi-Fi. The design combines hardware and embedded-software requirements so that the system is measurable, testable, scalable and suitable for a practical embedded-systems implementation.

2. Introduction

Environmental monitoring is a common embedded-system application because it requires continuous sensing, signal processing, decision making, user feedback and reliable communication. The proposed system is intended for a room, laboratory, small office or smart-home environment where users need an early indication of uncomfortable or unsafe conditions. The design emphasizes low-cost components, deterministic local response and optional cloud connectivity.

2.1 Problem Statement

Conventional standalone meters often provide only local readings and do not automatically react to abnormal conditions. A connected embedded monitor should collect multiple sensor inputs, filter noisy measurements, apply threshold logic, provide immediate local alerts, and communicate status remotely while remaining energy-conscious and fault-tolerant.

2.2 Objectives

- Define clear functional and non-functional requirements before implementation.
- Measure environmental parameters at a controlled sampling interval.
- Provide local visual and audible indication of normal and abnormal conditions.
- Control an external ventilation/exhaust load through a relay interface when required.
- Transmit telemetry to a cloud/dashboard service using a standard IoT protocol.
- Detect sensor or communication faults and enter a safe operating state.
- Create an architecture that can be expanded with additional sensors or actuators.

3. Methodology

The design follows a requirements-to-architecture workflow: identify the use case, convert user expectations into measurable requirements, map each requirement to hardware/software resources, define interfaces and timing, design the block diagram, identify failure modes, and prepare a verification plan. This prevents component selection from occurring before the required system behavior is understood.

3.1 Design Assumptions

- The system is installed indoors and powered from a regulated DC adapter.
- The controller has integrated Wi-Fi and sufficient GPIO/ADC/I²C resources.
- Environmental measurements do not require laboratory-grade calibration.
- The relay is used only through an electrically isolated driver/module suitable for the intended load.
- Cloud connectivity is an enhancement; essential alarms remain functional without the internet.

4. System Requirements Analysis

4.1 Functional Requirements

ID	Requirement	Priority	Verification
FR-01	Acquire temperature and humidity from DHT22 at least once every 5 s.	High	Functional test
FR-02	Acquire air-quality indicator and ambient-light information periodically.	High	Functional test
FR-03	Detect motion events using PIR input.	Medium	Functional test
FR-04	Show current readings and system state on OLED.	High	Inspection + test
FR-05	Activate buzzer when configured safety thresholds are crossed.	High	Threshold test
FR-06	Control exhaust/fan relay according to configured rules.	High	Actuator test
FR-07	Publish telemetry to a dashboard over Wi-Fi using MQTT/HTTP.	Medium	Communication test
FR-08	Log or indicate sensor, Wi-Fi and power-related faults.	High	Fault injection
FR-09	Recover automatically from transient communication failure.	Medium	Recovery test
FR-10	Allow threshold values to be configured in firmware or remotely.	Medium	Configuration test

4.2 Non-Functional Requirements

Category	Requirement / Target
Performance	Local alarm response target ≤ 1 s after a validated threshold violation is detected.
Sampling	Nominal environmental sampling period: 2–5 s; faster GPIO interrupt response for motion.
Availability	Local monitoring and alarms shall continue during temporary internet loss.
Reliability	Use input validation, timeout handling and watchdog/reset strategy where appropriate.
Power	Target low-power operation; sensor/communication duty cycle can be reduced when continuous reporting is unnecessary.
Maintainability	Modular drivers and configuration constants; clear interfaces between sensing, logic and communication.
Scalability	Architecture shall support additional I ² C/UART/GPIO sensors without redesigning the core control logic.
Security	Use authenticated Wi-Fi where supported and protected cloud credentials; do not expose secrets in source code.
Environmental	Indoor operating conditions; component ratings must match expected temperature, humidity and supply ranges.

5. Hardware Requirements

Component	Role	Key interface / reason
ESP32-class MCU	Central processing, control, timing and connectivity	GPIO, ADC, I ² C, timers, Wi-Fi; integrated connectivity reduces external parts.
DHT22	Temperature and relative humidity	Single digital data line; suitable for basic environmental monitoring.
MQ-135 module	Relative air-quality / gas indicator	Analog output to ADC; requires calibration and warm-up considerations.
LDR + resistor	Ambient-light sensing	Voltage divider to ADC; simple and inexpensive.
PIR sensor	Motion detection	Digital GPIO input; can generate event-driven activity.
0.96-in OLED	Local status and readings	I ² C; low pin count and readable local feedback.
Buzzer	Immediate audible alert	GPIO through suitable driver if required.
Relay module	Switch ventilation/exhaust load	GPIO control; use rated and isolated module for load.
5 V DC adapter + regulator	Power source and voltage conversion	Regulated rails; protection against over-current/reverse polarity as appropriate.

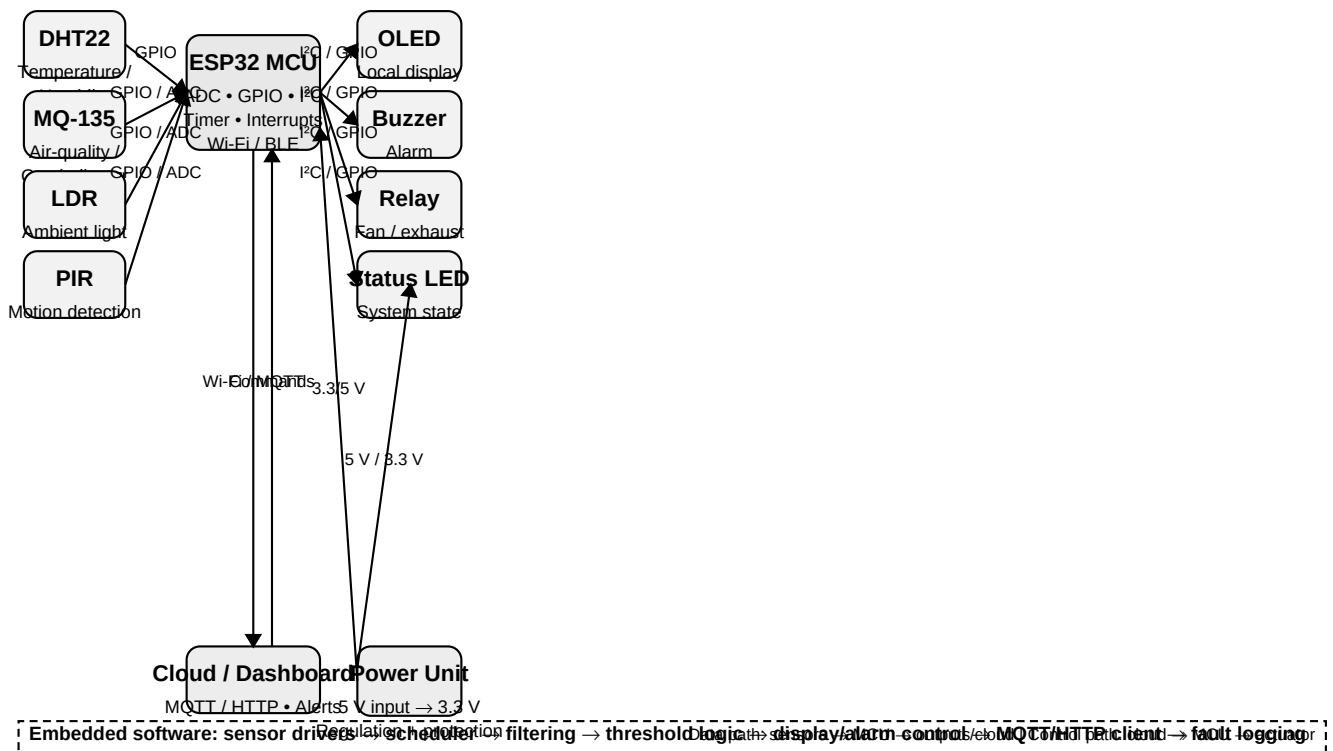
5.1 Input / Output Specifications

Signal	Direction	Typical form	Purpose
DHT22 DATA	Input	Digital	Temperature/humidity measurement
MQ-135 AO	Input	Analog	Air-quality indicator
LDR divider	Input	Analog	Ambient light level
PIR OUT	Input	Digital	Motion event
OLED SDA/SCL	Bidirectional	I ² C	Display data/control
Buzzer	Output	Digital/PWM	Local audible warning
Relay IN	Output	Digital	Ventilation/exhaust control
Wi-Fi	Bidirectional	2.4 GHz wireless	Cloud telemetry and optional commands

6. Comprehensive System Block Diagram

The architecture separates sensing, processing, user feedback, actuation, connectivity and power. Local decision-making is intentionally independent of the cloud so that safety-related alerts remain available during network outages.

Comprehensive Hardware–Software Block Diagram



Legend: Sensor data enters the MCU through digital/analog interfaces. The MCU performs validation, filtering, threshold evaluation and scheduling. Outputs drive the OLED, buzzer and relay. Wi-Fi provides telemetry and optional remote control. The power subsystem supplies regulated rails to the controller and peripherals.

7. Hardware Architecture Explanation

7.1 Sensor Layer

The sensor layer converts physical environmental conditions into electrical signals. Digital sensors are read through GPIO/protocol drivers, while analog sensors are sampled through the MCU ADC. Sensor readings should be validated for range, timeout and plausibility before being used by control logic.

7.2 Processing and Control Layer

The ESP32-class MCU is the system coordinator. It schedules sampling, executes filtering, maintains threshold states, updates the display, controls actuators, manages Wi-Fi/MQTT communication and records faults. A watchdog can be used to recover from software deadlock or unexpected execution stalls.

7.3 Local User Interface

The OLED provides immediate visibility of temperature, humidity, air-quality index/ADC value, motion state, Wi-Fi status and alarm state. A buzzer provides an attention-grabbing alert. Status indicators should clearly distinguish normal, warning and fault states.

7.4 Actuation Layer

A relay module can switch a low-voltage fan or an appropriately rated external exhaust system. The firmware should include hysteresis to avoid rapid relay toggling near a threshold. The relay interface must be selected for the actual load voltage/current and electrical safety requirements.

7.5 Connectivity Layer

Wi-Fi is used for remote telemetry. MQTT is preferred for lightweight publish/subscribe communication, while HTTP can be used where a REST endpoint is required. Communication tasks should have bounded timeouts and must not block critical local monitoring for long periods.

8. Embedded Software Architecture

Layer / Module	Responsibilities
Boot & Initialization	GPIO, ADC, I ² C, sensor initialization, configuration load, watchdog setup.
Sensor Drivers	Read sensors, handle timing, validate raw data and return standardized measurements.
Signal Processing	Range checks, moving average/median filtering, calibration offsets and derived values.
Scheduler / Main Loop	Coordinates periodic sampling, display refresh, communication and housekeeping.
Decision Engine	Applies thresholds, hysteresis, alarm state machine and actuator rules.
UI Manager	Formats values, status icons/messages and fault indications for OLED.
Communication Manager	Wi-Fi connection, MQTT/HTTP publish, retry/backoff and optional command reception.
Fault Manager	Sensor timeout, invalid data, network failure, watchdog and safe-state handling.

8.1 Representative Main-Loop Sequence

1. Start system and verify peripherals.
2. Read/validate sensors according to their timing requirements.
3. Filter noisy values and update derived measurements.
4. Evaluate warning/alarm thresholds with hysteresis.
5. Update OLED and local indicators.
6. Apply relay/fan control if the operating rule is satisfied.
7. Publish telemetry if the communication interval is due.
8. Check communication and sensor health; service watchdog.
9. Repeat continuously.

9. Communication Interface Design

Interface	Use	Design consideration
GPIO	PIR, buzzer, relay, digital sensor	Configure pull-up/down correctly; debounce or event filtering where necessary.
ADC	MQ-135 and LDR	Use stable reference/attenuation settings; calibrate and filter readings.
I ² C	OLED	Short wiring preferred; verify address and pull-up requirements.
Wi-Fi	Cloud connectivity	Reconnect with backoff; protect credentials.
MQTT	Telemetry / commands	Use topics for telemetry and state; QoS chosen according to reliability needs.

10. Power Management

A regulated DC supply is selected to provide stable voltage to the controller and peripherals. The power path should include appropriate protection, decoupling capacitors and a regulator capable of handling Wi-Fi current peaks. Sensors and the display can be duty-cycled where application requirements allow.

- Input: nominal 5 V regulated adapter for prototype use.
- Logic rail: regulated 3.3 V for ESP32 and compatible peripherals.

- Separate or well-decoupled supply path for relay/actuator loads to reduce electrical noise.
- Add bulk and local bypass capacitors close to the MCU and wireless module.
- Use brownout detection and controlled restart behavior where supported.
- For battery operation, introduce deep-sleep scheduling and wake-on-event strategies.

11. Real-Time and Timing Requirements

Task	Target interval / deadline	Priority rationale
PIR event handling	Event-driven; local indication within ~1 s	Immediate user response.
Temperature/humidity sampling	2–5 s	Environmental values change slowly.
Analog sensor sampling	1–5 s	Filtering improves stability.
OLED refresh	1–2 s	Human-readable update rate.
Alarm evaluation	After every validated sample	Must not be delayed by cloud operations.
Cloud telemetry	5–30 s configurable	Reduces network overhead.
Health/watchdog check	≤ 1–2 s	Detect stalled tasks promptly.

A key real-time design principle is that network communication must not block sensor acquisition or local alarm logic. The system can queue telemetry and retry in the background while local control continues.

12. Design Choice Justification

ESP32-class MCU: Combines sufficient processing, GPIO/ADC/I²C resources and Wi-Fi in one device, reducing board complexity.

DHT22: Provides both temperature and humidity through a simple digital interface and is adequate for a prototype-level monitor.

MQ-135: Offers an inexpensive analog indication for experimentation; readings must be treated as relative indicators unless calibrated.

LDR: Very low-cost method for ambient light detection and easy integration with an ADC.

PIR: Event-oriented motion sensing allows useful occupancy/activity detection without complex processing.

OLED over I²C: Readable local UI with only two signal lines, leaving MCU pins available for expansion.

MQTT: Lightweight publish/subscribe protocol suited to intermittent IoT telemetry and remote monitoring.

Local alarm logic: Maintains essential functionality when Wi-Fi or cloud services are unavailable.

13. Data Flow and Operating Logic

Stage	Operation	Output
1. Sense	Sensors capture physical conditions.	Raw digital/analog measurements
2. Acquire	MCU reads values at scheduled intervals.	Timestamped samples
3. Validate	Check range, timeout and plausibility.	Valid sample / fault flag
4. Filter	Apply simple moving average/median where needed.	Stable measurement
5. Decide	Compare against thresholds and hysteresis bands.	Normal / warning / alarm state
6. Act	Update buzzer, OLED and relay.	Local response
7. Publish	Send selected values/status to dashboard.	Remote telemetry
8. Recover	Retry failed communications and flag persistent faults.	Resilient operation

14. Fault Handling and Safety

Fault	Detection	Recommended response
DHT22 timeout/invalid data	Driver timeout/range check	Mark reading invalid; retain last-known-good value briefly; display fault if persistent.
Analog sensor out of range	ADC bounds / plausibility check	Ignore abnormal sample; flag sensor fault after repeated failures.
Wi-Fi unavailable	Connection state / timeout	Continue local monitoring; retry using increasing delay.
MQTT/HTTP failure	Publish timeout / error code	Queue latest telemetry; retry without blocking alarm logic.
MCU software stall	Watchdog	Automatic reset; initialize outputs to safe states.
Relay failure	Optional feedback/current sensing	Fail to safe state and report maintenance condition.
Power instability	Brownout/reset detection	Controlled restart; investigate supply sizing and decoupling.

Safety note: Any mains-powered appliance connected through a relay must use an appropriately rated, enclosed and electrically safe switching arrangement. The prototype design should preferably demonstrate with a low-voltage load.

15. Testing and Validation Plan

Test ID	Test	Expected result
T01	Power-on and initialization	MCU boots; peripherals initialize; outputs start in safe states.
T02	Temperature/humidity reading	Values appear on OLED and remain within plausible range.
T03	Air-quality analog response	ADC reading changes predictably with controlled sensor stimulus.
T04	Light-level response	ADC reading changes as illumination changes.
T05	Motion detection	PIR event updates system state and optional alert.
T06	Alarm threshold	Buzzer/relay activate only after valid threshold violation.
T07	Hysteresis	Relay does not chatter when measurement oscillates near threshold.
T08	Wi-Fi loss	Local monitoring and alarm continue; reconnection occurs automatically.
T09	Cloud publish	Telemetry reaches server/dashboard when network is available.
T10	Sensor fault injection	Fault is detected and clearly indicated without system crash.
T11	Watchdog recovery	Intentional software stall results in controlled reset and safe startup.

16. 30–35 Hour Work Breakdown

The following engineering effort allocation demonstrates the intended depth of work behind the submission. It represents analysis, design, documentation and validation planning rather than claiming that all activities required physical prototype assembly.

Activity	Estimated hours
Application research and use-case definition	3.0
Functional requirement analysis	4.0
Non-functional, timing and power requirements	3.0
Component/interface study	4.0
Architecture and block-diagram design	4.0
Embedded software architecture and data-flow planning	3.0
Fault handling and safety analysis	2.5

Activity	Estimated hours
Testing/verification plan	3.0
Technical documentation and report preparation	4.0
Review, refinement and consistency checking	2.5
Total	33.0 hours

17. Scalability and Future Enhancements

- Add CO₂, particulate matter (PM2.5/PM10), barometric pressure or VOC sensors.
- Replace basic thresholding with adaptive baselines or lightweight machine-learning anomaly detection.
- Add battery management and deep-sleep operation for portable deployments.
- Introduce secure device provisioning and encrypted MQTT/TLS communication.
- Add OTA firmware updates with rollback protection.
- Use a structured cloud database and dashboard for historical trends.
- Add multiple nodes and gateway architecture for larger buildings.
- Introduce calibration procedures and sensor-health metrics for improved measurement quality.

18. Limitations

- MQ-135-type modules are not a substitute for a calibrated professional air-quality analyzer.
- DHT22 is adequate for educational/prototype monitoring but has slower response and lower accuracy than industrial sensors.
- Wi-Fi availability and cloud services can affect remote monitoring, but should not disable local safety logic.
- Relay switching of mains loads requires appropriate electrical isolation, enclosure, protection and qualified handling.
- Actual power consumption depends strongly on Wi-Fi duty cycle, sensor modules and connected actuators.

19. Conclusion

The proposed IoT-Based Smart Environmental Monitoring and Alert System demonstrates a complete requirements-driven approach to embedded-system design. The architecture maps physical sensors to an ESP32-class controller, local user interfaces, actuation and remote connectivity while maintaining clear timing, power, fault-handling and safety requirements. The block diagram and supporting analysis provide a practical foundation for moving from conceptual design to schematic, firmware implementation, prototype assembly and system validation. The design is intentionally modular so that additional sensors, communication methods and cloud services can be introduced without changing the fundamental control architecture.

20. References

1. Espressif Systems, ESP32 Series technical documentation and datasheets.
2. DHT22/AM2302 sensor technical documentation and application notes.
3. MQ-135 gas sensor module documentation and calibration guidance.
4. SSD1306 OLED controller documentation / I²C interface references.
5. MQTT Version 3.1.1 / 5.0 protocol specifications.
6. Embedded-systems design principles covering watchdogs, ADC sampling, real-time scheduling and fault handling.
7. Component datasheets should be consulted during implementation to confirm exact electrical limits and module-specific pinouts.

End of Week 1 Submission

System Requirements Analysis and Block Diagram Design • IoT-Based Smart Environmental Monitoring and Alert System